

JAIDEVEDUCATIONSOCIETY'S
JDCOLLEGE OF ENGINEERING & MANAGEMENT,
NAGPUR



Department of Artificial Intelligence

SESSION: 2026-2027

3rd SEM-CSE(DS)

Subject: Data Structure using Python
Lab

List of Practical's

Semester/Branch:- III Sem/CSE(DS)

Subject Code:- DS3L002

Subject Name:- Data Structure using Python Lab

Subject In-charge: Mr. Samir Mendhe

Sr. No.	Name of Practicals
0	Know Your Lab and study Python features, data types, operators, and control flow statements.
1	Write a program to understand Python building blocks, data types, and basic input-output operations by creating a student data system.
2	To implement arithmetic, relational, logical, bitwise, and assignment operators in Python.
3	To perform various operations on Python lists and use built-in list functions.
4	To perform operations on tuple, set, and dictionary data structures.
5	To create a user-defined module and use built-in Python modules for mathematical and numeric operations.
6	To perform array creation, indexing, slicing, reshaping, and mathematical operations using NumPy.
7	To implement arrays and perform linear search and binary search in Python
8	To implement linked list(Singly, Doubly, Circular) structures in Python.
9	To implement stack operations and convert infix expression to postfix.
10	To manage tasks using stack and queue operations.
11	To simulate folder/file structure using binary tree.

Practical Incharge

Dept. Academic Incharge

Dept. Head CSE(DS)



PRACTICAL 0

Aim: Know Your Lab and study Python features, data types, operators, and control flow statements.

Theory:

1. Know Your Lab (Computer / Programming Lab Basics)

Before coding in Python, you should be familiar with your lab setup:

- **System:** Desktop/Laptop with Python installed (preferably Python 3.x)
- **IDE/Editor:** VS Code, PyCharm, or IDLE
- **Interpreter:** Python shell or terminal (python or python3)
- **File extension:** .py
- **Basic workflow:**
 1. Write code in editor
 2. Save file
 3. Run using terminal or run button
 4. Debug errors from output

2. Features of Python

Python is popular because it is:

- **Simple & readable** – close to English syntax
- **Interpreted** – runs line by line
- **Dynamically typed** – no need to declare variable types
- **Object-oriented** – supports classes and objects
- **Cross-platform** – works on Windows, Linux, macOS
- **Extensive libraries** – built-in modules for many tasks
- **Free & open-source**

3. Python Data Types

Basic Data Types

Type	Example	Description
int	10	Whole numbers
float	3.14	Decimal numbers
str	"Hello"	Text
bool	True, False	Logical values

Collection Types

Type	Example	Description
list	[1, 2, 3]	Ordered, changeable
tuple	(1, 2, 3)	Ordered, unchangeable
set	{1, 2, 3}	Unordered, unique values



Type	Example	Description
dict	<code>{"a": 1}</code>	Key-value pairs

4. Python Operators

Arithmetic Operators

- $+$ Addition $\rightarrow 5 + 2 = 7$
- $-$ Subtraction $\rightarrow 5 - 2 = 3$
- $*$ Multiplication $\rightarrow 5 * 2 = 10$
- $/$ Division $\rightarrow 5 / 2 = 2.5$
- $//$ Floor division $\rightarrow 5 // 2 = 2$
- $\%$ Modulus $\rightarrow 5 \% 2 = 1$
- $**$ Power $\rightarrow 2 ** 3 = 8$

Comparison Operators

- $==$ equal
- $!=$ not equal
- $>$ greater than
- $<$ less than
- $>=$ greater or equal
- $<=$ less or equal

Logical Operators

- and $\rightarrow$ both true
- or $\rightarrow$ at least one true
- not $\rightarrow$ reverse condition

Assignment Operators

- $=, +=, -=, *=, /=$

Example:

```
x = 5
x += 2    # x becomes 7
```

5. Control Flow Statements

Control flow decides how a program runs.

1. If Statement

```
x = 10
```



```
if x > 5:  
    print("Greater than 5")
```

2. If-Else

```
x = 3  
  
if x % 2 == 0:  
    print("Even")  
else:  
    print("Odd")
```

3. If-Elif-Else

```
marks = 75  
  
if marks >= 90:  
    print("A Grade")  
elif marks >= 60:  
    print("B Grade")  
else:  
    print("C Grade")
```

4. Loops

For Loop

```
for i in range(5):  
    print(i)
```

While Loop

```
i = 0  
while i < 5:  
    print(i)  
    i += 1
```

5. Loop Control Statements

- break → stops loop
- continue → skips current iteration
- pass → does nothing (placeholder)

Example:

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)
```

Conclusion: Hence we have successfully studied the different design techniques of algorithm



and complexity of algorithm.

PRACTICAL 1

Aim: Write a program to understand Python building blocks, data types, and basic input-output operations by creating a student data system.

Theory:

Python is a simple, interpreted, and high-level programming language widely used for developing applications. A Python program is built using basic elements called **building blocks**, which include variables, data types, operators, input/output functions, and expressions.

In this experiment, a **Student Data System** is created to take student information from the user, process it, and display the results.



1. Python Building Blocks

Python programs are made up of:

- **Variables:** Used to store data values
Example: `name = "Rahul"`
- **Keywords:** Reserved words like `if`, `else`, `print`
- **Expressions:** Combination of variables and operators
Example: `marks >= 40`
- **Statements:** Instructions executed by Python
Example: `print("Hello")`
- **Comments:** Used for explanation and ignored by interpreter
Example: `# This is a comment`

2. Data Types in Python

Python supports multiple data types:

◆ Basic Data Types

- **int** → Integer values (e.g., 10, 25)
- **float** → Decimal values (e.g., 78.5)
- **str** → Text or string values (e.g., "John")
- **bool** → Logical values (True/False)

◆ Example in Program:

- Name → `str`
- Roll number → `int`
- Marks → `float`
- Result (Pass/Fail) → `bool`

3. Input and Output Operations

◆ Input Function

- `input()` is used to take data from the user.
- It always returns data in string format unless converted.

Example:

```
name = input("Enter name: ")
```

◆ Output Function

- `print()` is used to display output on the screen.

Example:

```
print("Student Name:", name)
```

4. Processing Logic Used

The program performs simple processing such as:

- Accepting student details
- Converting input into required data types
- Calculating pass/fail condition using logical expression:

```
marks >= 40
```

```
Displaying formatted results
```

Program:

```
# Student Data System
```

```
# Demonstrates Python building blocks, data types, and I/O
```

```
print("=== Student Data System ===")
```

```
# Input section (user provides data)
```

```
name = input("Enter student name: ") # str
```

```
roll_no = int(input("Enter roll number: ")) # int
```

```
age = int(input("Enter age: ")) # int
```

```
marks = float(input("Enter marks: ")) # float
```

```
is_pass = marks >= 40 # bool (logical expression)
```



JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR
Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in
(An Autonomous Institute, with NAAC "A" Grade)
Affiliated to DBATU, RTMNU & MSBTE Mumbai
Department of Artificial Intelligence
"A Place to Learn, A Chance to Grow"
Session: 2026-27



```
# Processing section
percentage = (marks / 100) * 100

# Output section
print("\n=== Student Details ===")
print("Name:", name)
print("Roll Number:", roll_no)
print("Age:", age)
print("Marks:", marks)
print("Percentage:", percentage)
print("Result:", "Pass" if is_pass else "Fail")
```

```
# Display data types (to understand building blocks)
print("\n=== Data Types Used ===")
print("Type of name:", type(name))
print("Type of roll_no:", type(roll_no))
print("Type of age:", type(age))
print("Type of marks:", type(marks))
print("Type of is_pass:", type(is_pass))
```

Output:

```
=== Student Data System ===
Enter student name: raj
Enter roll number: 12
Enter age: 20
Enter marks: 50

=== Student Details ===
Name: raj
Roll Number: 12
Age: 20
Marks: 50.0
Percentage: 50.0
Result: Pass

=== Data Types Used ===
Type of name: <class 'str'>
Type of roll_no: <class 'int'>
Type of age: <class 'int'>
Type of marks: <class 'float'>
Type of is_pass: <class 'bool'>
```

Conclusion: Hence we have successfully understood Python building blocks, data types, and basic input-output operations by creating a student data system.



PRACTICAL 2

Aim: To implement arithmetic, relational, logical, bitwise, and assignment operators in Python.

Theory:

1. Introduction

Operators in Python are special symbols used to perform operations on variables and values. Python provides a rich set of built-in operators that help in performing mathematical calculations, comparisons, logical decision-making, bit-level operations, and value assignments. These operators form the foundation of programming logic and are widely used in problem-solving.

2. Types of Operators

2.1 Arithmetic Operators

Arithmetic operators are used to perform mathematical operations such as addition, subtraction, multiplication, division, etc.

Operator	Description	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
//	Floor Division	$a // b$
%	Modulus	$a \% b$
**	Exponent	$a ** b$

2.2 Relational (Comparison) Operators

Relational operators are used to compare two values. The result is always a Boolean value (True or False).

Operator	Description	Example
==	Equal to	$a == b$
!=	Not equal to	$a != b$
>	Greater than	$a > b$
<	Less than	$a < b$

Operator	Description	Example
----------	-------------	---------

>=	Greater than or equal to	a >= b
----	--------------------------	--------

<=	Less than or equal to	a <= b
----	-----------------------	--------

2.3 Logical Operators

Logical operators are used to combine conditional statements.

Operator	Description	Example
and	True if both conditions are true	a > 5 and b < 10
or	True if at least one condition is true	a > 5 or b < 10
not	Reverses the result	not(a > 5)

2.4 Bitwise Operators

Bitwise operators work on binary representations of numbers.

Operator	Description	Example
&	Bitwise AND	a & b
	Bitwise OR	
^	Bitwise XOR	a ^ b
~	Bitwise NOT	~a
<<	Left Shift	a << 1
>>	Right Shift	a >> 1

2.5 Assignment Operators

Assignment operators are used to assign values to variables and perform operations simultaneously.

Operator	Description	Example
=	Assign	a = 5
+=	Add and assign	a += 3
-=	Subtract and assign	a -= 2
*=	Multiply and assign	a *= 2
/=	Divide and assign	a /= 2
%=	Modulus and assign	a %= 2
//=	Floor divide and assign	a //= 2
**=	Exponent and assign	a **= 2

3. Working Principle

- Arithmetic operators perform mathematical calculations on numeric data.
- Relational operators compare values and return Boolean results.
- Logical operators combine multiple conditions to form complex decision-making expressions.



JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR
Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in
(An Autonomous Institute, with NAAC "A" Grade)
Affiliated to DBATU, RTMNU & MSBTE Mumbai
Department of Artificial Intelligence
"A Place to Learn, A Chance to Grow"
Session: 2026-27



- Bitwise operators manipulate data at the binary level, making them useful in low-level programming.
- Assignment operators simplify updating variable values by combining operations with assignment.

Program:

```
a = 10
b = 5

# Arithmetic
print("Addition:", a + b)
print("Division:", a / b)

# Relational
print("a > b:", a > b)

# Logical
print("a > 5 and b < 10:", a > 5 and b < 10)

# Bitwise
print("a & b:", a & b)

# Assignment
a += b
print("Updated a:", a)

# Membership
list1 = [1, 2, 3, 4]
print("2 in list1:", 2 in list1)

# Identity
x = 10
y = 10
print("x is y:", x is y)
```

Conclusion: Hence we have successfully implemented arithmetic, relational, logical, bitwise, and assignment operators in Python.



PRACTICAL 3

Aim: To perform various operations on Python lists and use built-in list functions.

Theory:

1. Introduction

A list in Python is a built-in data structure that is used to store multiple items in a single variable. Lists are **ordered, mutable (changeable), and allow duplicate values**. They are one of the most commonly used data types because they can store elements of different data types such as integers, strings, and even other lists.

Python provides a wide range of operations and built-in functions to manipulate lists efficiently.

2. List Operations

2.1 Creating a List

A list is created by placing elements inside square brackets [].

Example:

```
my_list = [10, 20, 30, 40]
```

2.2 Accessing Elements

List elements are accessed using indexing.

- Positive indexing starts from 0
- Negative indexing starts from -1 (last element)

Example:



```
print(my_list[0])    # First element
print(my_list[-1])   # Last element
```

2.3 Slicing of Lists

Slicing is used to access a range of elements.

```
print(my_list[1:3])  # Elements from index 1 to 2
```

2.4 Updating a List

Lists are mutable, so elements can be changed.

```
my_list[1] = 50
```

2.5 Adding Elements

Elements can be added using methods like `append()` and `insert()`.

```
my_list.append(60)    # Adds at end
my_list.insert(1, 25)  # Inserts at index 1
```

2.6 Removing Elements

Elements can be removed using `remove()`, `pop()`, or `clear()`.

```
my_list.remove(20)    # Removes specific value
my_list.pop()         # Removes last element
my_list.clear()       # Removes all elements
```

2.7 Looping through a List

Lists can be traversed using loops.

```
for item in my_list:
    print(item)
```

3. Built-in List Functions

Python provides several built-in functions to work with lists:

Function	Description
<code>len(list)</code>	Returns number of elements
<code>max(list)</code>	Returns largest element
<code>min(list)</code>	Returns smallest element
<code>sum(list)</code>	Returns sum of elements
<code>sorted(list)</code>	Returns sorted list
<code>list()</code>	Converts other data types to list

Example:



```
numbers = [5, 2, 9, 1]
```

```
print(len(numbers))
print(max(numbers))
print(min(numbers))
print(sum(numbers))
print(sorted(numbers))
```

4. List Methods

Common list methods include:

Method	Description
--------	-------------

append(x)	Adds element at end
insert(i, x)	Inserts at position i
remove(x)	Removes first occurrence of x
pop(i)	Removes element at index i
sort()	Sorts list in ascending order
reverse()	Reverses list

Program:

```
# Creating a list
numbers = [10, 20, 30, 40, 50]

# Accessing values
print("First element:", numbers[0])
print("Last element:", numbers[-1])

# Updating list
numbers[2] = 35
print("Updated list:", numbers)

# Adding elements
numbers.append(60)
numbers.insert(1, 15)
print("After insertion:", numbers)

# Deleting elements
numbers.remove(40)
del numbers[0]
print("After deletion:", numbers)

# Built-in functions
print("Length:", len(numbers))
print("Max:", max(numbers))
print("Min:", min(numbers))
numbers.sort()
print("Sorted list:", numbers)
```



Conclusion: Hence we have successfully performed various operations on Python lists and use built-in list functions.

PRACTICAL 4

Aim: To perform operations on tuple, set, and dictionary data structures.

Theory:

1. Introduction

Python provides several built-in data structures to store and organize data efficiently. Among them, **tuples, sets, and dictionaries** are widely used for different purposes.

Each data structure has unique properties:

- **Tuple** → Ordered and immutable collection



- **Set** → Unordered collection of unique elements
- **Dictionary** → Key-value pair based collection

These structures help in managing data effectively and are essential in real-world programming applications.

2. Tuple Data Structure

2.1 Definition

A tuple is an **ordered and immutable** collection of elements. Once created, its values cannot be changed.

2.2 Creating a Tuple

```
t = (10, 20, 30, 40)
```

2.3 Operations on Tuple

- **Accessing elements**

```
print(t[0])
```

- **Slicing**

```
print(t[1:3])
```

- **Concatenation**

```
t2 = (50, 60)
```

```
print(t + t2)
```

- **Repetition**

```
print(t * 2)
```

2.4 Key Feature

Tuples are immutable, meaning elements cannot be modified after creation.

3. Set Data Structure

3.1 Definition

A set is an **unordered collection of unique elements**. It does not allow duplicate values.

3.2 Creating a Set

```
s = {10, 20, 30, 40}
```

3.3 Operations on Set

- **Adding elements**

```
s.add(50)
```

- **Removing elements**

```
s.remove(20)
```

- **Union**



$s1 \mid s2$

- **Intersection**

$s1 \& s2$

- **Difference**

$s1 - s2$

3.4 Key Feature

Sets automatically remove duplicate values and support mathematical set operations.

4. Dictionary Data Structure

4.1 Definition

A dictionary is an **unordered collection of key-value pairs**. Each key is unique and used to access its corresponding value.

4.2 Creating a Dictionary

```
d = {"name": "Amit", "age": 20}
```

4.3 Operations on Dictionary

- **Accessing values**

```
print(d["name"])
```

- **Adding/updating elements**

```
d["city"] = "Nagpur"
```

- **Removing elements**

```
d.pop("age")
```

- **Iterating through dictionary**

```
for key, value in d.items():  
    print(key, value)
```

4.4 Key Feature

Dictionaries provide fast data retrieval using unique keys.

5. Comparison of Data Structures

Feature	Tuple	Set	Dictionary
Order	Ordered	Unordered	Unordered (Python 3.7+ maintains insertion order)
Duplicates	Allowed	Not allowed	Keys not allowed to repeat
Mutability	Immutable	Mutable	Mutable
Structure	Index-based	Value-based	Key-value pairs

Program:

```
# Tuple operations  
t = (10, 20, 30, 40)  
print("Tuple element:", t[1])
```



JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR
Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in
(An Autonomous Institute, with NAAC "A" Grade)
Affiliated to DBATU, RTMNU & MSBTE Mumbai
Department of Artificial Intelligence
"A Place to Learn, A Chance to Grow"
Session: 2026-27



```
# Set operations
s = {1, 2, 3, 4}
s.add(5)
s.remove(2)
print("Set:", s)

# Dictionary operations
student = {"name": "Amit", "age": 20, "marks": 85}

# Accessing values
print("Name:", student["name"])

# Updating values
student["marks"] = 90

# Adding new key
student["city"] = "Nagpur"

# Deleting key
del student["age"]

print("Updated Dictionary:", student)

# Built-in functions
print("Dictionary keys:", student.keys())
print("Dictionary values:", student.values())
print("Dictionary items:", student.items())
```

Conclusion: Hence we have successfully performed operations on tuple, set, and dictionary data structures.



PRACTICAL 5

Aim: To create a user-defined module and use built-in Python modules for mathematical and numeric operations.

Theory:

1. Introduction

In Python, a **module** is a file containing Python code (functions, variables, and classes) that can be reused in other programs. Modules help in organizing code, improving readability, and promoting reusability.

Python provides two types of modules:

- **Built-in modules** (predefined by Python)
- **User-defined modules** (created by programmers)

Built-in modules like `math`, `random`, and `statistics` are widely used for mathematical and numeric operations, while user-defined modules allow programmers to create their own reusable functionality.

2. User-Defined Modules

2.1 Definition

A user-defined module is a Python file (with `.py` extension) created by the user containing reusable functions and code.

2.2 Creating a Module

A module is created by saving Python code in a file.

Example:

```
# mymodule.py
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b
```

2.3 Using a User-Defined Module

The module is imported using the `import` keyword.

```
import mymodule

print(mymodule.add(5, 3))
print(mymodule.multiply(4, 2))
```

3. Built-in Python Modules

Python provides several built-in modules for mathematical and numeric operations.

3.1 Math Module

The `math` module provides mathematical functions.

Function	Description
<code>math.sqrt(x)</code>	Square root
<code>math.pow(x, y)</code>	Power
<code>math.factorial(x)</code>	Factorial
<code>math.ceil(x)</code>	Rounds up
<code>math.floor(x)</code>	Rounds down

Example:

```
import math

print(math.sqrt(25))
print(math.factorial(5))
print(math.ceil(4.3))
```

3.2 Random Module

The `random` module is used to generate random numbers.

Function	Description
<code>random()</code>	Random float between 0 and 1
<code>randint(a, b)</code>	Random integer between a and b
<code>choice(list)</code>	Random element from a list

Example:

```
import random

print(random.randint(1, 10))
print(random.choice([10, 20, 30]))
```

3.3 Statistics Module

The `statistics` module provides functions for statistical calculations.

Function	Description
<code>mean()</code>	Average
<code>median()</code>	Middle value
<code>mode()</code>	Most frequent value

Example:



```
import statistics

data = [10, 20, 20, 30]

print(statistics.mean(data))
print(statistics.median(data))
print(statistics.mode(data))
```

4. Working Principle

- User-defined modules allow programmers to create reusable code blocks.
 - Built-in modules provide ready-made functions for complex mathematical and statistical operations.
 - The `import` statement is used to access both types of modules in a program.
-

5. Advantages

- Promotes code reusability
- Reduces redundancy
- Improves program organization
- Saves development time
- Provides access to powerful built-in functionalities

Program:

Program 1: Built-in Module (math)

```
import math

num = 16

print("Square root:", math.sqrt(num))
print("Factorial:", math.factorial(5))
print("Power:", math.pow(2, 3))
print("Log:", math.log(10))
```

Program 2: Functional Programming Module

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
```



```
result = reduce(lambda x, y: x + y, numbers)
print("Sum using reduce:", result)
```

Program 3: User-defined Module Concept

Step 1: Create file `my_module.py`

```
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b
```

Step 2: Import and use module

```
import my_module

print("Addition:", my_module.add(5, 3))
print("Multiplication:", my_module.multiply(4, 2))
```

Conclusion: Hence we have successfully created a user-defined module and use built-in Python modules for mathematical and numeric operations.



Aim: To perform array creation, indexing, slicing, reshaping, and mathematical operations using NumPy.

Theory:

1. Introduction

NumPy (Numerical Python) is a powerful Python library used for scientific and numerical computing. It provides support for **multi-dimensional arrays** and offers a wide range of mathematical functions to perform efficient operations on large datasets.

The main object in NumPy is the **ndarray (N-dimensional array)**, which is faster and more efficient than Python lists for numerical computations.

2. Array Creation in NumPy

Arrays in NumPy can be created using different methods.

2.1 Creating Arrays from Lists

```
import numpy as np
```

```
arr = np.array([10, 20, 30, 40])
```

2.2 Creating Special Arrays

- Zeros array:

```
np.zeros(5)
```

- Ones array:

```
np.ones(4)
```

- Range-based array:

```
np.arange(1, 10, 2)
```

3. Indexing in NumPy Arrays

Indexing is used to access specific elements in an array.

```
arr = np.array([10, 20, 30, 40])
```

```
print(arr[0])    # First element  
print(arr[-1])   # Last element
```

For 2D arrays:

```
arr2 = np.array([[1, 2], [3, 4]])
```

```
print(arr2[0, 1]) # Row 0, Column 1
```

4. Slicing of NumPy Arrays

Slicing is used to access a range of elements.



```
arr = np.array([10, 20, 30, 40, 50])

print(arr[1:4]) # Elements from index 1 to 3
For 2D arrays:
```

```
arr2 = np.array([[1, 2, 3], [4, 5, 6]])

print(arr2[:, 1]) # All rows, column 1
```

5. Reshaping of Arrays

Reshaping is used to change the shape of an array without changing its data.

```
arr = np.array([1, 2, 3, 4, 5, 6])

reshaped = arr.reshape(2, 3)
```

6. Mathematical Operations in NumPy

NumPy allows element-wise mathematical operations.

6.1 Basic Operations

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b) # Addition
print(a - b) # Subtraction
print(a * b) # Multiplication
print(a / b) # Division
```

6.2 Built-in Mathematical Functions

Function	Description
----------	-------------

np.sum()	Sum of elements
np.mean()	Average
np.max()	Maximum value
np.min()	Minimum value
np.sqrt()	Square root

Example:

```
arr = np.array([1, 4, 9, 16])

print(np.sqrt(arr))
print(np.sum(arr))
```

Program:

Program 1: Array Creation

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
```




```
print("Array:", arr)
```

Program 2: Special Initialization

```
import numpy as np

print("Zeros:\n", np.zeros((2,2)))
print("Ones:\n", np.ones((3,3)))
print("Full:\n", np.full((2,2), 7))
print("Random:\n", np.random.rand(2,2))
```

Program 3: Indexing, Slicing, Reshaping

```
import numpy as np

arr = np.array([[1,2,3],
                [4,5,6],
                [7,8,9]])

# Indexing
print("Element:", arr[1][2])

# Slicing
print("Slice:\n", arr[0:2, 1:3])

# Reshaping
new_arr = arr.reshape(1, 9)
print("Reshaped:", new_arr)
```

Program 4: Array Operations

```
import numpy as np

a = np.array([1,2,3])
b = np.array([4,5,6])

print("Addition:", a + b)
print("Multiplication:", a * b)
print("Mean:", np.mean(a))
print("Sum:", np.sum(b))
```

Conclusion: Hence we have successfully performed array creation, indexing, slicing, reshaping,



and mathematical operations using NumPy.

PRACTICAL 7

Aim: To implement arrays and perform linear search and binary search in Python

Theory:

1. Introduction

An array is a linear data structure that stores elements in **contiguous memory locations** and allows efficient access using indices. In Python, arrays can be implemented using **lists** or the array module, but lists are most commonly used due to their flexibility.

Searching is a fundamental operation in data structures used to locate an element within a collection. Two common searching techniques are:

- **Linear Search**
- **Binary Search**

2. Arrays in Python

In Python, arrays are typically represented using lists.



Example of Array (List)

arr = [10, 20, 30, 40, 50]

Features of Arrays

- Store multiple elements of same or different types
 - Indexed structure (starts from 0)
 - Supports traversal, insertion, deletion, and searching
-

3. Linear Search

3.1 Definition

Linear search is a simple searching technique where each element of the array is checked **sequentially** until the target element is found or the list ends.

3.2 Working Principle

- Start from the first element
- Compare each element with the target value
- Stop when element is found or end of array is reached

3.3 Algorithm

1. Start from index 0
2. Compare current element with key
3. If match found, return index
4. Else move to next element
5. Repeat until end of array

3.4 Example

arr = [10, 20, 30, 40, 50]

key = 30

for i in range(len(arr)):

if arr[i] == key:

print("Element found at index", i)

break

3.5 Time Complexity

- Best case: $O(1)$
 - Worst case: $O(n)$
-

4. Binary Search

4.1 Definition

Binary search is an efficient searching technique used on **sorted arrays**. It works by repeatedly dividing the search interval in half.

4.2 Working Principle

- Compare key with middle element
 - If equal $\rightarrow$ element found
 - If key is smaller $\rightarrow$ search left half
 - If key is larger $\rightarrow$ search right half
 - Repeat until found or interval becomes empty
-

4.3 Algorithm



1. Set low = 0, high = n-1
 2. Find mid = (low + high) // 2
 3. Compare middle element with key
 4. If match → return index
 5. If key < mid element → search left half
 6. Else → search right half
 7. Repeat until low <= high
-

4.4 Example

arr = [10, 20, 30, 40, 50]

key = 30

low = 0

high = len(arr) - 1

while low <= high:

mid = (low + high) // 2

if arr[mid] == key:

print("Element found at index", mid)

break

elif arr[mid] < key:

low = mid + 1



else:

high = mid - 1

4.5 Time Complexity

- Best case: $O(1)$
 - Worst case: $O(\log n)$
-

5. Comparison Between Linear and Binary Search

Feature	Linear Search	Binary Search
Data Requirement	Unsorted or sorted	Must be sorted
Approach	Sequential	Divide and conquer
Time Complexity	$O(n)$	$O(\log n)$
Efficiency	Slow for large data	Fast for large data

Program:

Program 1: Linear Search

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 25, 30, 45, 60]
key = 30

result = linear_search(arr, key)

if result != -1:
    print("Element found at index:", result)
else:
```

```
print("Element not found")
```

Program 2: Binary Search

```
def binary_search(arr, key):  
    low = 0  
    high = len(arr) - 1  
  
    while low <= high:  
        mid = (low + high) // 2  
  
        if arr[mid] == key:  
            return mid  
        elif arr[mid] < key:  
            low = mid + 1  
        else:  
            high = mid - 1  
  
    return -1  
  
arr = [10, 20, 30, 40, 50]  
key = 40  
  
print("Index:", binary_search(arr, key))
```

Conclusion: Hence we have successfully implemented arrays and perform linear search and binary search in Python

PRACTICAL 8

Aim: To implement linked list(Singly, Doubly, Circular) structures in Python.

1. Introduction

A linked list is a linear data structure in which elements (called nodes) are stored in separate memory locations and connected using pointers (references). Unlike arrays, linked lists do not require contiguous memory allocation, making them more flexible for dynamic memory management.

Each node in a linked list contains:

- Data part → stores the value
- Pointer (link) → stores reference to the next (or previous) node

Linked lists are widely used in applications where frequent insertion and deletion are required.

2. Singly Linked List



2.1 Definition

A singly linked list is a type of linked list in which each node points only to the next node in the sequence.

2.2 Structure

Each node contains:

[Data | Next]

2.3 Working Principle

- The first node is called the head
- The last node points to NULL
- Traversal is possible only in one direction

2.4 Operations

- Insertion (at beginning, end, or position)
- Deletion
- Traversal

3. Doubly Linked List

3.1 Definition

A doubly linked list is a linked list in which each node contains two pointers:

- One pointing to the next node
- One pointing to the previous node

3.2 Structure

[Prev | Data | Next]

3.3 Working Principle

- Traversal is possible in both forward and backward directions
- Requires more memory due to an extra pointer

3.4 Operations

- Forward traversal
- Backward traversal
- Insertion and deletion at both ends

4. Circular Linked List

4.1 Definition

A circular linked list is a variation of a linked list in which the last node points back to the first node, forming a circular structure.

4.2 Structure

Head → Node → Node → Node → Head

4.3 Types

- Singly circular linked list
- Doubly circular linked list

4.4 Working Principle

- No NULL at the end of the list
- Traversal continues until we reach the starting node again

5. Comparison of Linked Lists



Feature	Singly Linked List	Doubly Linked List	Circular Linked List
Direction	One-way	Two-way	Circular
Memory Usage	Less	More	Moderate
Traversal	Forward only	Forward & backward	Continuous loop
Last Node	Points to NULL	Points to NULL	Points to head

Program:

A) Singly Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

l1 = LinkedList()
l1.insert(10)
l1.insert(20)
l1.insert(30)

l1.display()
```

(B) Doubly Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
```

(C) Circular Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```



JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR
Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in
(An Autonomous Institute, with NAAC "A" Grade)
Affiliated to DBATU, RTMNU & MSBTE Mumbai
Department of Artificial Intelligence
"A Place to Learn, A Chance to Grow"
Session: 2026-27



```
class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)

        if not self.head:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = new_node
            new_node.next = self.head

    def display(self):
        temp = self.head
        if self.head:
            while True:
                print(temp.data, end=" -> ")
                temp = temp.next
                if temp == self.head:
                    break
            print("(back to head)")

c11 = CircularLinkedList()
c11.insert(5)
c11.insert(10)
c11.insert(15)

c11.display()
```

Conclusion: Hence we have successfully implemented linked list(Singly, Doubly, Circular) structures in Python.



PRACTICAL 9

Aim: To implement stack operations and convert infix expression to postfix.

Theory:

1. Introduction

A **stack** is a linear data structure that follows the **LIFO (Last In First Out)** principle, meaning the last element inserted is the first one to be removed. Stack is widely used in programming for expression evaluation, function calls, recursion, and syntax parsing. In this experiment, we study:

- Basic stack operations
- Conversion of **infix expressions to postfix expressions** using a stack

2. Stack Data Structure

2.1 Definition

A stack is an ordered collection of elements where insertion and deletion take place from only one end called the **TOP**.

2.2 Stack Operations

Operation	Description
Push	Insert element at top
Pop	Remove element from top
Peek/Top	View top element
isEmpty	Check if stack is empty

2.3 Working Principle

- Stack follows **LIFO principle**
- Only the top element is accessible at any time
- Overflow occurs when stack is full



- Underflow occurs when stack is empty

3. Stack Implementation (Conceptual in Python)

Stacks can be implemented using:

- Lists
- `append()` → Push operation
- `pop()` → Pop operation

Example:

```
stack = []
```

```
stack.append(10)
```

```
stack.append(20)
```

```
print(stack.pop())
```

4. Expression Notations

4.1 Infix Expression

Operators are written between operands.

Example:

$A + B * C$

4.2 Postfix Expression (Reverse Polish Notation)

Operators are written after operands.

Example:

$A B C * +$

5. Conversion of Infix to Postfix Using Stack

5.1 Purpose

Infix expressions are difficult for computers to evaluate directly due to precedence rules.

Postfix expressions eliminate the need for parentheses and are easier to evaluate.

5.2 Algorithm

1. Initialize an empty stack and output string
2. Scan the infix expression from left to right
3. If operand → add to output
4. If '(' → push to stack
5. If ')' → pop until '(' is found
6. If operator:
 - Pop operators with higher or equal precedence

- Push current operator
7. Pop all remaining operators from stack to output

5.3 Operator Precedence

Operator Precedence

^	Highest
* / %	Medium
+ -	Lowest

5.4 Example

Infix:

A + B * C

Postfix:

A B C * +

6. Advantages of Stack in Expression Conversion

- Simplifies expression evaluation
- Eliminates ambiguity of precedence
- Efficient for compiler design
- Easy implementation using arrays/lists

7. Applications of Stack

- Expression evaluation
- Syntax parsing in compilers
- Undo/Redo operations
- Backtracking algorithms
- Function call management (recursion)

Program:

Program 1: Stack Implementation

```
stack = []

# Push operation
stack.append(10)
stack.append(20)
stack.append(30)
```



```
print("Stack:", stack)

# Pop operation
stack.pop()
print("After pop:", stack)

# Peek
print("Top element:", stack[-1])
```

Program 2: Infix to Postfix (Simple)

```
def precedence(op):
    if op == '+' or op == '-':
        return 1
    if op == '*' or op == '/':
        return 2
    return 0

def infix_to_postfix(expression):
    stack = []
    result = ""

    for char in expression:
        if char.isalnum():
            result += char
        elif char == '(':
            stack.append(char)
        elif char == ')':
            while stack and stack[-1] != '(':
                result += stack.pop()
            stack.pop()
        else:
            while stack and precedence(stack[-1]) >=
precedence(char):
                result += stack.pop()
            stack.append(char)

    while stack:
        result += stack.pop()

    return result

expr = "A+B*C"
print("Postfix:", infix_to_postfix(expr))
```



Conclusion: Hence we have successfully implemented stack operations and convert infix expression to postfix.

PRACTICAL 10

Aim: To manage tasks using stack and queue operations.

Theory:

1. Introduction

Data structures play an important role in organizing and managing data efficiently. Two commonly used linear data structures for task management are **Stack** and **Queue**. They help in handling tasks based on specific ordering rules.

- A **Stack** follows the **LIFO (Last In First Out)** principle.
- A **Queue** follows the **FIFO (First In First Out)** principle.

These structures are widely used in real-life applications such as scheduling, process management, and memory handling.

2. Stack for Task Management

2.1 Definition

A stack is a linear data structure in which elements are inserted and removed from one end called the **TOP**.

2.2 Operations

Operation	Description
Push	Add a task to stack
Pop	Remove the most recent task
Peek	View top task
isEmpty	Check if stack is empty

2.3 Working Principle

- The most recently added task is executed first.
- Useful in scenarios like undo operations or backtracking tasks.



2.4 Example

Tasks are added in order:

Task A → Task B → Task C

Execution order (LIFO):

Task C → Task B → Task A

3. Queue for Task Management

3.1 Definition

A queue is a linear data structure in which elements are inserted at the **rear** and removed from the **front**.

3.2 Operations

Operation	Description
Enqueue	Add a task to queue
Dequeue	Remove the oldest task
Front	View first task
isEmpty	Check if queue is empty

3.3 Working Principle

- The first task added is executed first.
 - Used in real-world scheduling systems like CPU scheduling, print queue, etc.
-

3.4 Example

Tasks are added in order:

Task A → Task B → Task C

Execution order (FIFO):

Task A → Task B → Task C

4. Comparison Between Stack and Queue

Feature	Stack	Queue
Order	LIFO	FIFO
Insertion	Top only	Rear
Deletion	Top only	Front
Usage	Undo operations, recursion	Scheduling, buffering

5. Applications in Task Management

- **Stack Applications**



- Undo/Redo operations in software
- Backtracking problems
- Function call management
- **Queue Applications**
 - CPU scheduling
 - Print job management
 - Customer service systems
 - Data buffering in networks

6. Advantages

- Efficient task organization
- Simple implementation
- Helps in scheduling and process management
- Reduces complexity in task handling systems

Program:

```
# =====  
# TASK MANAGEMENT SYSTEM  
# STACK + QUEUE IMPLEMENTATION  
# =====
```

```
from collections import deque
```

```
# Queue for task scheduling (FIFO)  
task_queue = deque()
```

```
# Stack for undo operations (LIFO)  
undo_stack = []
```

```
def add_task(task):  
    task_queue.append(task)  
    print(f"Task added: {task}")
```

```
def process_task():  
    if task_queue:  
        task = task_queue.popleft()  
        undo_stack.append(task)  
        print(f"Processed task: {task}")  
    else:  
        print("No tasks to process")
```



JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR
Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in
(An Autonomous Institute, with NAAC "A" Grade)
Affiliated to DBATU, RTMNU & MSBTE Mumbai
Department of Artificial Intelligence
"A Place to Learn, A Chance to Grow"
Session: 2026-27



```
def undo_task():
    if undo_stack:
        task = undo_stack.pop()
        print(f"Undo last processed task: {task}")
    else:
        print("Nothing to undo")

def show_tasks():
    print("\nPending Tasks (Queue):", list(task_queue))
    print("Undo Stack:", undo_stack)

# =====
# MENU DRIVEN PROGRAM
# =====
while True:
    print("\n===== TASK MANAGER =====")
    print("1. Add Task")
    print("2. Process Task")
    print("3. Undo Task")
    print("4. Show Tasks")
    print("5. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        task = input("Enter task: ")
        add_task(task)

    elif choice == "2":
        process_task()

    elif choice == "3":
        undo_task()

    elif choice == "4":
        show_tasks()

    elif choice == "5":
        print("Exiting system...")
        break

    else:
        print("Invalid choice")
```



Conclusion: Hence we have successfully managed tasks using stack and queue operations.

PRACTICAL 11

Aim: To simulate folder/file structure using binary tree.

Theory:

1. Introduction

A **binary tree** is a hierarchical data structure in which each node can have at most two children, known as the **left child** and **right child**. It is widely used to represent hierarchical relationships in computing systems.

A real-world example of a hierarchical system is a **folder/file structure in an operating system**, where folders contain files and subfolders. This structure can be efficiently modeled using a tree data structure.

2. Binary Tree Representation

2.1 Definition

A binary tree is a collection of nodes where each node consists of:

- Data (folder or file name)
- Left child pointer
- Right child pointer

2.2 Structure

Each node can be represented as:

```
      Folder
     /      \
Subfolder  File
```

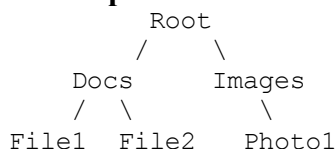
3. Folder/File System Using Binary Tree

3.1 Concept

In this simulation:

- Each **node** represents a folder or file.
- The **root node** represents the main directory.
- The **left child** can represent a subfolder or first child item.
- The **right child** can represent another subfolder or file at the same level.

3.2 Example Structure



4. Operations on Binary Tree

4.1 Insertion

New files or folders are added as left or right children of a node.

4.2 Traversal

Traversal helps in viewing the structure:

- **Inorder (Left → Root → Right)**
- **Preorder (Root → Left → Right)**
- **Postorder (Left → Right → Root)**

Example:

Preorder traversal: Root → Docs → File1 → File2 → Images → Photo1

4.3 Searching

Search operation is used to find a specific file or folder in the tree.

4.4 Deletion

A node (file/folder) can be removed from the tree while maintaining structure.

5. Advantages of Using Binary Tree

- Represents hierarchical data efficiently
 - Easy navigation between parent and child nodes
 - Faster searching compared to linear structures (in some cases)
 - Useful for structured data representation
-



6. Limitations

- Limited to two children per node (not ideal for complex file systems)
- Can become unbalanced, affecting performance
- More complex implementation than arrays or lists

7. Applications

- File and directory systems
- Expression trees in compilers
- Database indexing
- Hierarchical organization of data
- Decision-making systems

Program:

```
# =====  
# FILE SYSTEM SIMULATOR USING TREE  
# BINARY SEARCH TREE IMPLEMENTATION  
# =====
```

```
class Node:
```

```
    def __init__(self, name):  
        self.name = name  
        self.left = None  
        self.right = None
```

```
class FileSystem:
```

```
    def __init__(self):  
        self.root = None  
  
    # Insert node (BST logic)  
    def insert(self, root, name):  
        if root is None:  
            return Node(name)  
  
        if name < root.name:  
            root.left = self.insert(root.left, name)  
        else:  
            root.right = self.insert(root.right, name)  
  
        return root
```

```
    # Search node  
    def search(self, root, name):  
        if root is None:  
            return False
```



JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR
Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in
(An Autonomous Institute, with NAAC "A" Grade)
Affiliated to DBATU, RTMNU & MSBTE Mumbai
Department of Artificial Intelligence
"A Place to Learn, A Chance to Grow"
Session: 2026-27



```
if root.name == name:
    return True
elif name < root.name:
    return self.search(root.left, name)
else:
    return self.search(root.right, name)

# Inorder traversal
def inorder(self, root):
    if root:
        self.inorder(root.left)
        print(root.name, end=" ")
        self.inorder(root.right)

# Preorder traversal
def preorder(self, root):
    if root:
        print(root.name, end=" ")
        self.preorder(root.left)
        self.preorder(root.right)

# Postorder traversal
def postorder(self, root):
    if root:
        self.postorder(root.left)
        self.postorder(root.right)
        print(root.name, end=" ")

# =====
# MAIN PROGRAM
# =====
fs = FileSystem()
root = None

while True:
    print("\n===== FILE SYSTEM MENU =====")
    print("1. Insert File/Folder")
    print("2. Search File/Folder")
    print("3. Inorder Display")
    print("4. Preorder Display")
    print("5. Postorder Display")
    print("6. Exit")

    choice = input("Enter choice: ")

    if choice == "1":
        name = input("Enter file/folder name: ")
```



JAIDEV EDUCATION SOCIETY'S
J D COLLEGE OF ENGINEERING AND MANAGEMENT
KATOL ROAD, NAGPUR
Website: www.jdcoem.ac.in E-mail: info@jdcoem.ac.in
(An Autonomous Institute, with NAAC "A" Grade)
Affiliated to DBATU, RTMNU & MSBTE Mumbai
Department of Artificial Intelligence
"A Place to Learn, A Chance to Grow"
Session: 2026-27



```
root = fs.insert(root, name)
print("Inserted successfully")

elif choice == "2":
    name = input("Enter name to search: ")
    if fs.search(root, name):
        print("Found in system")
    else:
        print("Not found")

elif choice == "3":
    print("Inorder (Sorted View):")
    fs.inorder(root)

elif choice == "4":
    print("Preorder (Structure View):")
    fs.preorder(root)

elif choice == "5":
    print("Postorder View:")
    fs.postorder(root)

elif choice == "6":
    print("Exiting system...")
    break

else:
    print("Invalid choice")
```

Conclusion: Hence we have successfully simulated folder/file structure using binary tree.